

# PyTorch 实战工作流入门

PyTorch

是当前深度学习领域最主流的框架，掌握其完整工作流是进行AI研究和开发的基础。本指南从零开始，覆盖从

PyTorch 实战工作流入门

PyTorch

是当前深度学习领域最主流的框架，掌握其完整工作流是进行AI研究和开发的基础。本指南从零开始，覆盖从

- - -

## 一、PyTorch 安装与环境配置

### 1.1 安装指南

基础安装 (CPU)

```
pip install torch torchvision
```

CUDA 11.8 安装

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
```

CUDA 12.1 安装

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

验证安装

```
python -c "import torch; print(torch.version); print(torch.cuda.is_available())"
```

### 1.2 Conda环境管理 (推荐)

创建独立环境

```
conda create -n pytorchenv python=3.10
```

```
conda activate pytorchenv
```

安装PyTorch (conda方式, 自动处理CUDA依赖)

```
conda install pytorch torchvision pytorch-cuda=12.1
```

常用配套包

```
pip install tensorboard # 训练可视化
```

```
pip install tqdm # 进度条
pip install scikit-learn # 评估指标
pip install matplotlib # 绘图
pip install einops # 张量操作简化
```

### 1.3 框架对比

框架 定位 图类型 调试 部署 社区

PyTorch 科研首选 动态图 容易 TorchScript/ONNX 学术界主流

TensorFlow 工业部署 静态图(2.x兼容动态) 较难 TF Serving/TF Lite

JAX 高性能计算 函数式 中等 XLA 前沿研究

PaddlePaddle 国产框架 动态图 容易 Paddle Inference 国内工业界

### 1.4 可复现性设置

```
import torch
import numpy as np
import random

def seedeverything(seed=42):
    """ 固定所有随机种子，确保实验可复现 """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True # 牺牲一些速度
    torch.backends.cudnn.benchmark = False # 关闭自动优化

seedeverything(42)

- - -
```

## 二、张量(Tensor)详解

张量是PyTorch的核心数据结构，支持GPU加速和自动求导。

## 2.1 张量创建

```
import torch
import numpy as np
```

从Python数据创建

```
x = torch.tensor([1, 2, 3], dtype=torch.float32)
```

创建特定形状

```
xzeros = torch.zeros(3, 4) # 全0
xones = torch.ones(3, 4) # 全1
xrand = torch.randn(3, 4) # 标准正态分布
xrange = torch.arange(0, 10, 2) # 等差序列 [0, 2, 4, 6, 8]
xlinspace = torch.linspace(0, 1, 5) # 等间隔 [0, 0.2, 0.4, 0.6, 0.8]
xeye = torch.eye(3) # 单位矩阵
xfull = torch.full((2, 3), 7.0) # 填充特定值
```

从NumPy转换（共享内存！）

```
nparray = np.array([1, 2, 3])
xfromnp = torch.from_numpy(nparray) # NumPy → Tensor
xtonp = xfromnp.numpy() # Tensor → NumPy
```

从另一个张量

```
xlike = torch.randnlike(xzeros) # 相同形状的随机张量
```

## 2.2 张量属性

```
x = torch.randn(2, 3, 4)
```

```
print(x.shape) # torch.Size([2, 3, 4])
print(x.size()) # torch.Size([2, 3, 4])
print(x.dim()) # 3（维度数）
print(x.dtype) # torch.float32
print(x.device) # cpu 或 cuda:0
print(x.requires_grad) # 是否需要梯度
print(x.numel()) # 24（总元素数）
```

## 2.3 张量运算

### 算术运算

```
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([4.0, 5.0, 6.0])

z1 = x + y # 逐元素加 [5, 7, 9]
z2 = x * y # 逐元素乘 [4, 10, 18]
z3 = x ** 2 # 逐元素平方 [1, 4, 9]
z4 = torch.dot(x, y) # 点积 32
```

### 矩阵运算

```
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = A @ B # 矩阵乘法 (3, 5) — 推荐
C = torch.matmul(A, B) # 等价写法
```

### 聚合运算

```
x = torch.randn(3, 4)
x.sum() # 标量
x.sum(dim=0) # 沿第0维求和 → (4, )
x.mean(dim=1) # 沿第1维求均值 → (3, )
x.max(dim=1) # 返回(值, 索引)
x.argmax(dim=1) # 返回最大值的索引
```

### 形状操作

```
x = torch.randn(2, 3, 4)
x.reshape = x.reshape(6, 4) # 重塑(可共用内存)
x.view = x.view(6, 4) # 视图(必须连续)
x.permute = x.permute(2, 0, 1) # 维度交换 → (4, 2, 3)
x.transpose = x.transpose(0, 1) # 交换两个维度
x.squeeze = x.unsqueeze(0) # 增加维度 → (1, 2, 3, 4)
x.squeeze = x.squeeze.squeeze(0) # 移除大小为1的维度
```

### 广播(Broadcasting)

```
a = torch.ones(3, 1) # (3, 1)
b = torch.ones(1, 4) # (1, 4)
c = a + b # 自动广播为 (3, 4)
```

## 2.4 GPU 操作

### 检查GPU

```
print(torch.cuda.isavailable())
print(torch.cuda.devicecount())
print(torch.cuda.getdevicename(0))
```

### 设备选择

```
device = torch.device("cuda" if torch.cuda.isavailable())
```

### 张量移到GPU

```
x = torch.randn(3, 4)
x = x.to(device) # 移到GPU
x = x.cuda() # 简写(如果有GPU)
x = x.cpu() # 移回CPU
```

### 创建时指定设备

```
x = torch.randn(3, 4, device=device)
```

### 多GPU

```
model = torch.nn.DataParallel(model) # 数据并行
- - -
```

## 三、自动求导 (Autograd)

### 3.1 基本使用

#### 创建需要梯度的张量

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
```

#### 前向计算

```
y = x ** 2 # y = [4, 9]
```

```
z = y.sum() # z = 13
```

反向传播

```
z.backward()
```

查看梯度

```
print(x.grad) # tensor([4., 6.]) = dz/dx = 2x
```

为什么是 [4, 6] ?

$$dz/dx_1 = dz/dy_1 \cdot dy_1/dx_1 = 1 \cdot 2 \cdot 2 = 4$$
$$dz/dx_2 = dz/dy_2 \cdot dy_2/dx_2 = 1 \cdot 2 \cdot 3 = 6$$

### 3.2 关键注意事项

1. 梯度累积！每次backward会在grad上累加

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
for i in range(3):
    y = (x ** 2).sum()
    y.backward()
print(x.grad) # [4, 6] → [8, 12] → [12, 18] # 累积了！
```

解决：每次backward前清零

```
x.grad.zero()
```

2. 禁用梯度（推理时使用）

```
with torch.no_grad():
    output = model(input) # 不计算梯度，节省内存
```

3. detach()：分离张量与计算图

```
x = torch.randn(3, requires_grad=True)
y = x ** 2
z = y.detach() # z不再追踪梯度
```

4. item()：将标量张量转为Python数值

```
loss = criterion(output, target)
lossvalue = loss.item() # 转为float
```

- - -

## 四、数据集与数据加载器

### 4.1 自定义 Dataset

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    """自定义数据集模板"""
    def __init__(self, features, labels, transform=None):
        self.features = features
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        x = self.features[idx]
        y = self.labels[idx]

        if self.transform:
            x = self.transform(x)

        return x, y
```

使用

```
dataset = CustomDataset(Xtrain, ytrain)
```

### 4.2 图像数据集

```
from torchvision import datasets, transforms
```

数据增强 Pipeline

```
traintransform = transforms.Compose([
    transforms.Resize(256), # 缩放
    transforms.RandomCrop(224), # 随机裁剪
```

```

transforms.RandomHorizontalFlip(p=0.5), # 随机水平翻转
transforms.ColorJitter( # 颜色抖动
    brightness=0.2, contrast=0.2, saturation=0.2, hue
),
transforms.RandomRotation(10), # 随机旋转
transforms.ToTensor(), # 转为张量
transforms.Normalize( # 归一化
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
        ])

```

#### 加载数据集

```

traindataset = datasets.CIFAR10(
    root='./data', train=True, download=True, transform
)

```

#### 4.3 DataLoader 详解

```

trainloader = DataLoader(
    traindataset,
    batchsize=64, # 批量大小
    shuffle=True, # 训练集打乱
    numworkers=4, # 多进程加载(Windows下建议0)
    pinmemory=True, # 加速CPU→GPU传输
    drop_last=False, # 是否丢弃最后不完整的batch
    persistentworkers=True # 保持worker进程存活
)

```



迭代数据

```
for batchidx, (data, target) in enumerate(trainloader):
    data, target = data.to(device), target.to(device)
    # 训练代码...
```

获取一个batch

```
firstbatch = next(iter(trainloader))
print(f"数据形状: {firstbatch[0].shape}") # (64, 3, 28, 28)
print(f"标签形状: {firstbatch[1].shape}") # (64,)

- - -
```

## 五、构建神经网络

### 5.1 使用 nn.Module (推荐)

```
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self, inputdim, hiddendim, outputdim):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(inputdim, hiddendim)
        self.bn1 = nn.BatchNorm1d(hiddendim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(hiddendim, outputdim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = SimpleNet(784, 256, 10)
```

## 5.2 常用网络层

层 类名 说明

全连接 `nn.Linear(in, out)`  $y = Wx + b$

卷积 `nn.Conv2d(in_c, out_c, kernel)` 2D卷积

池化 `nn.MaxPool2d(kernel)` 最大池化

批归一化 `nn.BatchNorm1d/2d` 批归一化

Dropout `nn.Dropout(p)` 随机失活

嵌入 `nn.Embedding(num, dim)` 词嵌入

LSTM `nn.LSTM(input, hidden)` LSTM层

Transformer `nn.TransformerEncoder` Transformer编码器

## 5.3 CNN模型示例（图像分类）

```
class SimpleCNN(nn.Module):
    """轻量级CNN，适用于CIFAR-10等小图像"""
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1: 3 → 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2), # 32x32 → 16x16
            nn.Dropout2d(0.25),

            # Block 2: 32 → 64
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
```

```

nn.Conv2d(64, 64, kernel_size=3, padding=1),
nn.BatchNorm2d(64),
nn.ReLU(inplace=True),
nn.MaxPool2d(2, 2), # 16x16 → 8x8
nn.Dropout2d(0.25),

# Block 3: 64 → 128
nn.Conv2d(64, 128, kernel_size=3, padding=1),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.MaxPool2d(2, 2), # 8x8 → 4x4
nn.Dropout2d(0.25),
)
self.classifier = nn.Sequential(
nn.Linear(128 * 4 * 4, 256),
nn.ReLU(inplace=True),
nn.Dropout(0.5),
nn.Linear(256, num_classes)
)

def forward(self, x):
x = self.features(x)
x = x.view(x.size(0), -1) # 展平
x = self.classifier(x)
return x

class ResidualBlock(nn.Module):
"""ResNet残差块"""
def __init__(self, inchannels, outchannels, stride=1):
super().__init__()
self.conv1 = nn.Conv2d(inchannels, outchannels, kernel_size=3,
stride=stride, padding=1, bias=False)
self.bn1 = nn.BatchNorm2d(outchannels)
self.conv2 = nn.Conv2d(outchannels, outchannels, kernel_size=3,
padding=1, bias=False)

```

```

stride=1, padding=1, bias=False)
self.bn2 = nn.BatchNorm2d(outchannels)

self.shortcut = nn.Sequential()
if stride != 1 or inchannels != outchannels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(inchannels, outchannels, kernel_size=1,
        nn.BatchNorm2d(outchannels)
    )

def forward(self, x):
    out = F.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    out += self.shortcut(x) # 残差连接
    out = F.relu(out)
    return out

class ResNet18(nn.Module):
    """ 简化版ResNet-18 """
    def __init__(self, numclasses=10):
        super().__init__()
        self.inchannels = 64

        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)

        self.layer1 = self.makelayer(64, 2, stride=1)
        self.layer2 = self.makelayer(128, 2, stride=2)
        self.layer3 = self.makelayer(256, 2, stride=2)
        self.layer4 = self.makelayer(512, 2, stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, numclasses)

    def makelayer(self, outchannels, numblocks, stride):
        strides = [stride] + [1]*(numblocks - 1)

```

```

layers = []
for s in strides:
    layers.append(ResidualBlock(self.inchannels, outc
self.inchannels = outchannels
return nn.Sequential(layers)

def forward(self, x):
x = F.relu(self.bn1(self.conv1(x)))
x = self.layer1(x)
x = self.layer2(x)
x = self.layer3(x)
x = self.layer4(x)
x = self.avgpool(x)
x = x.view(x.size(0), -1)
x = self.fc(x)
return x

```

## 5.4 迁移学习

迁移学习是深度学习最重要的实践技巧之一——使用预训练模型微调，大幅减少训练数据和时间：

```
import torchvision.models as models
```

===== 方法1：特征提取（冻结预训练层） =====

适用：小数据集（< 1 K样本），只训练最后的分类头

加载预训练模型

```
model_ft = models.resnet18(weights=models.ResNet18
```

冻结所有参数

```

for param in model_ft.parameters():
    param.requires_grad = False

```

替换最后的全连接层

```

numfeatures = model_ft.fc.infeatures
model_ft.fc = nn.Linear(numfeatures, numclasses) #

```

只有 f c 层的参数会被更新

```
optimizer = torch.optim.Adam(model_ft.fc.parameters)
```

==== 方法2：微调（解冻部分或全部层）====

适用：中等数据集（1 K - 1 0 K 样本），调整预训练权重

```
model_ft2 = models.resnet18(weights=models.ResNet1
model_ft2.fc = nn.Linear(model_ft2.fc.in_features, n
```

分层学习率：底层用小 l r，顶层用大 l r

```
optimizer = torch.optim.Adam([
{'params': model_ft2.layer1.parameters(), 'lr': 1e
{'params': model_ft2.layer2.parameters(), 'lr': 1e
{'params': model_ft2.layer3.parameters(), 'lr': 5e
{'params': model_ft2.layer4.parameters(), 'lr': 1e
{'params': model_ft2.fc.parameters(), 'lr': 1e-3}],
lr=1e-3)
```

==== 方法3：渐进式解冻 ====

适用：大数据集，避免破坏预训练特征

阶段1：只训练 f c（1 - 2 个 e p o c h）

阶段2：解冻 l a y e r 4，训练 2 - 3 个 e p o c h

阶段3：解冻 l a y e r 3，继续训练

... 逐步解冻更多层

```
def gradual_unfreeze(model, epoch, unfreezeschedule):
"""
```

渐进式解冻

```
unfreezeschedule: {epochnum: [layernamestounfreeze]}
"""
```

```
if epoch in unfreezeschedule:
for name, param in model.named_parameters():
for layername in unfreezeschedule[epoch]:
if layername in name:
```

```
param.requires_grad = True
print(f"Unfreezing: {name}")
```

迁移学习最佳实践：

场景    数据量    方法    学习率    训练轮数

小数据+相似任务    < 1 K    特征提取     $1e-3$     10 - 20

小数据+不同任务    < 1 K    特征提取+中间层     $1e-3$  /  $1e-4$     20 - 30

中数据+相似任务    1 K - 10 K    微调     $1e-4$     10 - 20

中数据+不同任务    1 K - 10 K    渐进式解冻     $1e-4 \rightarrow 1e-5$     20 - 50

大数据    > 10 K    从头训练或全量微调     $1e-4$     50 - 100

## 5.5 模型信息查看

打印模型结构

```
print(model)
```

统计参数量

```
totalparams = sum(p.numel() for p in model.parameters())
trainableparams = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"总参数量: {totalparams:,}")
print(f"可训练参数量: {trainableparams:,}")
```

查看各层参数

```
for name, param in model.named_parameters():
    print(f"{name}: {param.shape}")

---
```

## 六、训练完整流程

### 6.1 标准训练循环

```
import torch
import torch.nn as nn
import torch.optim as optim
```

## 超参数

```
config = {  
    'learningrate': 0.001,  
    'batchsize': 64,  
    'numepochs': 20,  
    'weightdecay': 1e-4,  
}
```

### 1. 准备数据

```
trainloader = DataLoader(traindataset, batchsize=  
valloader = DataLoader(valdataset, batchsize=config
```

### 2. 创建模型

```
model = SimpleNet(784, 256, 10).to(device)
```

### 3. 定义损失函数和优化器

```
criterion = nn.CrossEntropyLoss()  
optimizer = optim.Adam(model.parameters(), lr=config['learningrate'],  
weightdecay=config['weightdecay'])
```

### 4. 学习率调度器

```
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, config['numepochs'])
```

### 5. 训练循环

```
bestvalacc = 0.0  
patiencecounter = 0  
patience = 5  
  
for epoch in range(config['numepochs']):  
    # === 训练阶段 ===  
    model.train()  
    trainloss = 0.0  
    traincorrect = 0  
    traintotal = 0
```



```

for batchX, batchy in trainloader:
    batchX, batchy = batchX.to(device), batchy.to(device)

    # 前向传播
    outputs = model(batchX)
    loss = criterion(outputs, batchy)

    # 反向传播
    optimizer.zero_grad() # 清空梯度
    loss.backward() # 计算梯度
    torch.nn.utils.clip_grad_norm(model.parameters(), max_norm)
    optimizer.step() # 更新参数

    # 统计
    trainloss += loss.item() * batchX.size(0)
    , predicted = outputs.max(1)
    traintotal += batchy.size(0)
    traincorrect += predicted.eq(batchy).sum().item()

    trainloss /= traintotal
    trainacc = 100 * traincorrect / traintotal

    # === 验证阶段 ===
    model.eval()
    valloss = 0.0
    valcorrect = 0
    valtotal = 0

    with torch.no_grad():
        for batchX, batchy in valloader:
            batchX, batchy = batchX.to(device), batchy.to(device)
            outputs = model(batchX)
            loss = criterion(outputs, batchy)

            valloss += loss.item() * batchX.size(0)
            , predicted = outputs.max(1)

```

```

valtotal += batchy.size(0)
valcorrect += predicted.eq(batchy).sum().item()

valloss /= valtotal
valacc = 100. * valcorrect / valtotal

# 学习率调度
scheduler.step()

# Early Stopping
if valacc > bestvalacc:
    bestvalacc = valacc
    patiencecounter = 0
    torch.save(model.state_dict(), 'bestmodel.pth')
else:
    patiencecounter += 1
    if patiencecounter >= patience:
        print(f"Early stopping at epoch {epoch+1}")
        break

print(f"Epoch {epoch+1} / {config['numepochs']} "
      f"Train Loss: {trainloss:.4f} Acc: {trainacc:.2f} "
      f"Val Loss: {valloss:.4f} Acc: {valacc:.2f}% "
      f"LR: {scheduler.getlastlr()[0]:.6f}")

- - -

```

## 七、混合精度训练

```

from torch.cuda.amp import GradScaler, autocast

scaler = GradScaler() # 梯度缩放器

for batchX, batchy in trainloader:
    batchX, batchy = batchX.to(device), batchy.to(device)

    optimizer.zero_grad()

```

```

# 前向传播使用 F P 1 6
with autocast():
    outputs = model(batchX)
    loss = criterion(outputs, batchy)

# 反向传播（自动缩放）
scaler.scale(loss).backward()
scaler.unscale(optimizer)
torch.nn.utils.clip_grad_norm(model.parameters(), n
scaler.step(optimizer)
scaler.update()

```

混合精度训练原理：

为什么混合精度能加速？

传统 F P 3 2 训练：

前向传播： F P 3 2 矩阵乘法 → F P 3 2 结果  
 反向传播： F P 3 2 梯度计算 → F P 3 2 参数更新  
 显存占用： 每个参数 4 字节 × N

混合精度训练：

前向传播： F P 1 6 矩阵乘法 → F P 1 6 结果（速度翻倍）  
 反向传播： F P 1 6 梯度计算 → F P 3 2 参数更新（精度保证）  
 显存占用： 激活值减半，参数仍 F P 3 2

关键：参数主副本保持 F P 3 2（master weights），  
 每步将 F P 3 2 权重转为 F P 1 6 做前向，梯度用 F P 1 6 计算后转回 F P 3 2 更新

GradScaler 的作用：

F P 1 6 的梯度可能下溢（太小变 0），scaler 将 loss 乘以一个大数，  
 使梯度放大到 F P 1 6 可表示范围，更新前再缩回来。

- - -

八、分布式训练

## 8.1 DataParallel vs DistributedDataParallel

DataParallel (DP):

单进程多线程 → 简单但有GIL瓶颈

GPU 0 负责收集/分发 → 负载不均衡

适合：快速验证、单机多卡

DistributedDataParallel (DDP):

多进程 → 无GIL瓶颈

每个GPU独立进程 → 负载均衡

Ring-AllReduce通信 → 高效

适合：生产训练、多机多卡

## 8.2 DDP 训练脚本

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel
from torch.utils.data.distributed import DistributedSampler

def setupddp():
    """初始化DDP"""
    dist.init_process_group(backend="nccl")
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    return local_rank

def main():
    local_rank = setupddp()
    device = torch.device(f"cuda:{local_rank}")

    # 模型包装
    model = SimpleNet(784, 256, 10).to(device)
    model = DDP(model, device_ids=[local_rank])

    # 数据分片（每个进程只看到部分数据）
    trainsampler = DistributedSampler(traindataset)
```

```

trainloader = DataLoader(
    traindataset,
    batchsize=64,
    sampler=train_sampler, # 替代shuffle
    numworkers=4,
    pinmemory=True,
)

for epoch in range(numepochs):
    train_sampler.set_epoch(epoch) # 确保每个epoch数据打乱不同
    for batchX, batchy in trainloader:
        batchX, batchy = batchX.to(device), batchy.to(device)
        # 正常训练...

dist.destroy_process_group()

```

启动命令：

```

torchrun --nprocpernode=4 trainddp.py (4卡训练)
torchrun --nprocpernode=2 --nnodes=2 --noderank=0

```

### 8.3 梯度累积（显存不足时的技巧）

当batchsize=32但显存只够batchsize=8时

```

accumulationsteps = 4 # 累积4步 = 等效batchsize=32

```

```

optimizer.zero_grad()
for i, (batchX, batchy) in enumerate(trainloader):
    outputs = model(batchX)
    loss = criterion(outputs, batchy) / accumulationsteps
    loss.backward() # 梯度累积

if (i + 1) % accumulationsteps == 0:
    torch.nn.utils.clip_grad_norm(model.parameters(), max_norm)
    optimizer.step()
    optimizer.zero_grad()

```

- - -

## 八、模型保存与加载

方式1：仅保存参数（推荐）

```
torch.save(model.state_dict(), 'modelparams.pth')
model = SimpleNet(784, 256, 10)
model.load_state_dict(torch.load('modelparams.pth'),
```

方式2：保存完整checkpoint

```
checkpoint = {
    'epoch': epoch,
    'modelstate_dict': model.state_dict(),
    'optimizerstate_dict': optimizer.state_dict(),
    'schedulerstate_dict': scheduler.state_dict(),
    'bestvalacc': bestvalacc,
}
torch.save(checkpoint, 'checkpoint.pth')
```

加载checkpoint

```
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['modelstate_dict'])
optimizer.load_state_dict(checkpoint['optimizerstate_dict'])
scheduler.load_state_dict(checkpoint['schedulerstate_dict'])
startepoch = checkpoint['epoch'] + 1
```

- - -

## 九、TensorBoard 可视化

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter('runs/experiment1')
```

记录损失

```
writer.add_scalar('Loss/train', trainloss, epoch)
writer.add_scalar('Loss/val', valloss, epoch)
```

记录准确率

```
writer.addscalar('Accuracy/train', trainacc, epoch)
writer.addscalar('Accuracy/val', valacc, epoch)
```

记录学习率

```
writer.addscalar('Learningrate', optimizer.paramg
```

记录模型图

```
writer.addgraph(model, inputsample)
```

记录图像

```
writer.addimages('sampleimages', images, epoch)
```

记录直方图（观察参数分布）

```
for name, param in model.namedparameters():
writer.addhistogram(name, param, epoch)
```

记录嵌入（高维向量可视化）

```
writer.addembedding(features, metadata=labels, la

writer.close()
```

启动TensorBoard

```
tensorboard --logdir=runs
```

实验管理最佳实践

多实验对比：每次实验用不同的子目录

```
writer = SummaryWriter(f'runs/lr{lr}bs{batchsize}
```

同时对比多个实验

```
tensorboard --logdir=runs ← 自动加载runs / 下所有子目录

- - -
```

十、高级调试技巧

## 10.1 梯度检查

验证自定义层的梯度是否正确

```
from torch.autograd import gradcheck

class MyCustomLayer(nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x) # Swish激活

layer = MyCustomLayer()
x = torch.randn(10, requires_grad=True, dtype=torch.float)

gradcheck会用数值梯度验证自动梯度
result = gradcheck(layer, x, eps=1e-6, atol=1e-4)
print(f"梯度检查: {'通过' if result else '失败'}")
```

## 10.2 常见训练问题排查

问题1：Loss不下降

检查清单：

- ☐ 学习率是否合适？（试试 $1e-3$ 或 $1e-4$ ）
- ☐ 数据是否正确加载？（打印一个batch看看）
- ☐ 标签是否正确？（分类任务检查标签范围）
- ☐ 梯度是否正常？（打印梯度范数）
- ☐ 是否忘记optimizer.step()？

问题2：Loss为NaN

检查清单：

- ☐ 学习率是否太大？（降低10倍）
- ☐ 数据是否有NaN/Inf？（np.isnan检查）
- ☐ 是否除以零？（加epsilon）
- ☐ log操作是否输入负数？（加clamp）
- ☐ 混合精度是否下溢？（检查GradScaler）

问题3：训练很慢

检查清单：



- ☐ 是否使用了 `DataLoader` 的 `numworkers` ?
- ☐ 是否使用了 `pinmemory=True` ?
- ☐ 是否使用了混合精度训练 ?
- ☐ 是否有频繁的 CPU - GPU 数据传输 ?
- ☐ 是否在每个 `batch` 都做了不必要的计算 ?

#### 梯度监控工具

```
def monitorgradients(model):
    """ 监控模型各层的梯度统计 """
    for name, param in model.namedparameters():
        if param.grad is not None:
            gradnorm = param.grad.norm().item()
            gradmean = param.grad.mean().item()
            gradstd = param.grad.std().item()
            if gradnorm > 100 or gradnorm < 1e-7:
                print(f"⚠ {name}: gradnorm={gradnorm:.6f} "
                      f"mean={gradmean:.6f} std={gradstd:.6f}")
            # 检查NaN
            if torch.isnan(param.grad).any():
                print(f" {name}: 梯度包含NaN!")
```

#### 在训练循环中调用

```
loss.backward()
monitorgradients(model)
optimizer.step()
```

### 10.3 PyTorch Profiler 性能分析

```
from torch.profiler import profile, recordfunction
```

#### 分析CPU和GPU操作

```
with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.GPU],
             profilememory=True, recordshapes=True) as prof:
    with recordfunction("modelinference"):
        output = model(inputdata)
```

打印耗时最多的操作

```
print(prof.key_averages().table(sortby="cuda_time_total"))
```

导出可视化（在Chrome浏览器查看 `chrome://tracing`）

```
prof.export_chrome_trace("trace.json")
```

常见性能瓶颈：

1. CPU→GPU数据传输过多 → 增大batch、pinmemory
2. 某个算子特别慢 → 检查是否有更快的实现
3. GPU利用率低 → 可能被CPU数据加载阻塞

- - -

## 十一、模型保存与加载

方式1：仅保存参数（推荐）

```
torch.save(model.state_dict(), 'model_params.pth')
model = SimpleNet(784, 256, 10)
model.load_state_dict(torch.load('model_params.pth'))
```

方式2：保存完整checkpoint（支持断点续训）

```
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict(),
    'best_val_acc': best_val_acc,
    'config': config, # 保存超参数
}
torch.save(checkpoint, 'checkpoint.pth')
```

加载checkpoint恢复训练

```
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
```

```

scheduler.load_state_dict(checkpoint['scheduler_state_dict'])
startepoch = checkpoint['epoch'] + 1
bestvalacc = checkpoint['bestvalacc']

- - -

```

## 十二、项目组织最佳实践

```

project /
├── config.py # 超参数配置
├── data /
│   ├── dataset.py # 自定义Dataset
│   └── preprocess.py # 数据预处理
├── models /
│   └── model.py # 模型定义
├── utils /
│   ├── metrics.py # 评估指标
│   └── utils.py # 工具函数
├── train.py # 训练脚本
├── evaluate.py # 评估脚本
├── inference.py # 推理脚本
├── requirements.txt # 依赖
└── README.md # 说明文档

```

### 超参数配置

```

from dataclasses import dataclass

@dataclass
class Config:
    # 数据
    datapath: str = "./data"
    batchsize: int = 64
    numworkers: int = 4

    # 模型

```

```

inputdim: int = 784
hiddendim: int = 256
outputdim: int = 10

# 训练
learningrate: float = 1e-3
weightdecay: float = 1e-4
numepochs: int = 50
patience: int = 5
gradclip: float = 1.0

# 设备
device: str = "cuda" if torch.cuda.isavailable()
seed: int = 42

def seedeverything(self):
    torch.manualseed(self.seed)
    np.random.seed(self.seed)
    if torch.cuda.isavailable():
        torch.cuda.manualseedall(self.seed)

- - -

```

### 十三、调试技巧

#### 13.1 过拟合 / 欠拟合诊断

现象   训练损失   验证损失   诊断   解决方案

欠拟合   高   高   模型太简单   增加容量、减少正则化

过拟合   低   高   模型太复杂   增加正则化、数据增强

训练失败   不下降   —   学习率 / 数据问题   降低  $l_r$ 、检查数据

震荡   震荡   震荡    $l_r$  太大   降低  $l_r$ 、增大 `batch`

#### 13.2 学习率选择

学习率查找器（简化版）

```

def findlr(model, trainloader, criterion, device,
optimizer = optim.SGD(model.parameters(), lr=initlr),
lrmult = (finallr / initlr) * (1 / nsteps)

lrs = []
losses = []

for i, (x, y) in enumerate(trainloader):
    if i >= nsteps:
        break

    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()

    lrs.append(optimizer.paramgroups[0]['lr'])
    losses.append(loss.item())
    optimizer.paramgroups[0]['lr'] = lrmult

return lrs, losses

```

使用方法：绘制 `lr vs loss` 曲线，选择 `loss` 下降最快处的 `lr`

- - -

#### 十四、HuggingFace Transformers 集成

PyTorch 在 NLP / LLM 领域通常与 HuggingFace Transformers

```

from transformers import AutoModelForSequenceClassification, TrainingArguments

```

加载预训练模型

```

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-chinese",

```

```
numlabels=2
```

```
)
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base
```

### 数据预处理

```
def tokenizefunction(examples):
```

```
    return tokenizer(examples["text"], padding="maxlen
```

```
tokenizeddatasets = dataset.map(tokenizefunction,
```

### 训练配置

```
trainingargs = TrainingArguments(
```

```
    outputdir=". / results",
```

```
    numtrainepochs=3,
```

```
    perdevicetrainbatchsize=16,
```

```
    perdeviceevalbatchsize=32,
```

```
    learningrate=2e-5,
```

```
    weightdecay=0.01,
```

```
    evaluationstrategy="epoch",
```

```
    savestrategy="epoch",
```

```
    loadbestmodelatend=True,
```

```
    fp16=True, # 混合精度
```

```
    gradientaccumulationsteps=2,
```

```
    loggingsteps=50,
```

```
    reportto="tensorboard",
```

```
)
```

### 训练

```
trainer = Trainer(
```

```
model=model,
```

```
args=trainingargs,
```

```
traindataset=tokenizeddatasets["train"],
```

```
evaldataset=tokenizeddatasets["validation"],
```

```
)
```

```
trainer.train()
```

推理

```
from transformers import pipeline
classifier = pipeline("text-classification", model=
result = classifier("这个产品非常好用")
print(result) # [{'label': 'POSITIVE', 'score': 0.9999999}

- - -
```

十五、einops——优雅的张量操作

```
pip install einops
from einops import rearrange, reduce, repeat
```

`rearrange`: 更直观的张量维度操作

将  $(b, c, h, w)$  转为  $(b, h * w, c)$

```
x = torch.randn(2, 3, 224, 224)
y = rearrange(x, 'b c h w -> b (h w) c') # 比 x.permute(0, 2, 3, 1) 更直观
```

将图像分成 patch (ViT 中使用)

```
patches = rearrange(x, 'b c (h p1) (w p2) -> b (h p1) (w p2) c'
(2, 3, 224, 224) → (2, 196, 768)
```

`reduce`: 降维操作

全局平均池化

```
avg = reduce(x, 'b c h w -> b c', 'mean') # 等价于 x.mean(dim=[2, 3])
```

`repeat`: 复制操作

将  $(b, c)$  扩展为  $(b, c, h, w)$

```
x_2d = torch.randn(2, 256)
x_4d = repeat(x_2d, 'b c -> b c h w', h=14, w=14)
```

- - -

核心总结：PyTorch 的完整工作流为：环境配置 → 张量操作 → 数据加载 → 模型构建 → 训练循环（含验证、学习率调度、早停）→ 保存加载 → 推理。进阶要点包括：混合精度训练（AMP）、分布式训练（DDP）、梯度累积、梯度裁剪、TensorBoard。

`Transformers` 提供了更高层的封装。掌握这套完整流程，就能快速搭建和训练任何深度学习模型，